Práctica 3: Métodos Actor-Critic para Control Continuo

Pablo García Molina[1], Antonio Lorenzo Díaz-Meco[1], and Andrés Martínez Fuentes[1]

[1]Universidad Pontificia Comillas – Máster MUIA, Aprendizaje por Refuerzo Profundo

Curso 2025/2026

1. Introducción

Esta práctica aborda el control continuo de un agente bípedo mediante métodos Actor-Critic. Se parametriza directamente la política $\pi_\theta(a|s)$ y se optimiza mediante dos algoritmos representativos: **PPO** (*Proximal Policy Optimization*), un método *on-policy*, y **SAC** (*Soft Actor-Critic*), un método *off-policy* basado en el principio de máxima entropía.

El entorno elegido es **Walker2D-v4**, accedido a través de la interfaz Gymnasium. El agente recibe únicamente imágenes RGB como observación, lo que obliga a aprender simultáneamente una representación visual del estado y una política de control sobre ella. Se mantiene el espacio de acciones continuo ($N_a = 6$ dimensiones) para ambos algoritmos, en concordancia con las fortalezas de PPO y SAC.

2. Configuración común del entorno

2.1. Espacio de acciones

Se utiliza un espacio de acciones **continuo** de $N_a = 6$ dimensiones correspondientes a los actuadores articulares del Walker2D. Cada componente toma valores en $[-1, 1]$ tras el escalado. Esta elección se mantiene idéntica para PPO y SAC, aprovechando que ambos algoritmos están diseñados para espacios de acción continuos y evitando sesgos de discretización.

2.2. Observación

La observación consiste exclusivamente en la imagen RGB renderizada del entorno. Cada fotograma se recorta eliminando un 25 % superior y un 5 % inferior de la imagen (además de un 10 % en cada lateral) para centrar la región de interés, y se redimensiona a 84×84 píxeles mediante interpolación de área (`INTER_AREA`). Se mantiene el color RGB (3 canales) para facilitar la distinción visual entre las piernas del agente.

Se aplica *frame stacking* de $k = 4$ fotogramas consecutivos, concatenados a lo largo del eje de canales, produciendo una observación de dimensiones $84 \times 84 \times 12$ en formato `uint8`.

2.3. Función de recompensa

Se utiliza la recompensa por defecto de `Walker2d-v4`, que combina avance positivo, un bono de supervivencia (*healthy bonus*) y una penalización por el coste de control. Opcionalmente, se aplica un *reward shaping* adicional mediante `WalkerRewardWrapper`, que añade un término proporcional a la velocidad en x:

$$r_{\text{shaped}} = r_{\text{original}} + 0{,}1 \cdot \dot{x}_{\text{torso}} \tag{1}$$

donde $\dot{x}_{\text{torso}}$ se obtiene del vector de velocidades generalizadas del modelo MuJoCo (`qvel[0]`). Para SAC se reduce además el rango angular de *healthy* a $[-0{,}4, 0{,}4]$ rad, penalizando inclinaciones excesivas.

3. Parte 1: Locomoción con PPO

3.1. Metodología

PPO optimiza la política mediante el objetivo subrogado con *clipping*:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\text{mín}\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] \tag{2}$$

con $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. Las ventajas se estiman mediante GAE:

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{3}$$

La función de pérdida total combina tres componentes:

$$L = L^{\text{CLIP}} + c_v \cdot L^{\text{value}} - c_e \cdot H[\pi] \tag{4}$$

con $c_v = 0{,}5$ (coeficiente del crítico) y c_e como coeficiente de entropía. Adicionalmente se emplea *value clipping* para estabilizar el entrenamiento del crítico, y un mecanismo de *early stopping* basado en la divergencia KL aproximada ($1{,}5 \times$ target_KL).

Arquitectura. El modelo es una red `CNNGaussianActorCritic` con actor y crítico compartiendo un *encoder* CNN:

- **Encoder CNN** (tipo Nature-DQN): `Conv2d`(12→32, 8×8, stride 4) → ReLU → `Conv2d`(32→64, 4×4, stride 2) → ReLU → `Conv2d`(64→64, 3×3, stride 1) → ReLU

- **Capa FC compartida**: `Linear`($n_{\text{flat}} \to 512$) $\to$ ReLU
- **Cabeza de política (actor)**: `Linear`($512 \to 6$) produce μ; la desviación típica σ es un parámetro libre (`nn.Parameter`, compartido entre estados), inicializado a $\log \sigma_0 = -1{,}0$ y acotado a $[-4{,}0,\ 0{,}5]$.
- **Cabeza de valor (crítico)**: `Linear`($512 \to 1$).

La acción media se escala mediante $\mu = \tanh(\mu_{\text{raw}}) \cdot s + b$, donde s y b son la *escala* y *bias* derivados de los límites del espacio de acción, asegurando que la media se mantenga dentro del rango válido. Todas las capas emplean inicialización ortogonal (ganancia $\sqrt{2}$ para las capas interiores, 0,01 para la cabeza de μ y 1,0 para la cabeza de valor).

Cuadro 1: Hiperparámetros de PPO empleados en el entrenamiento.

Hiperparámetro	Valor
Tasa de aprendizaje	10^{-4} (con *annealing* lineal)
ϵ (clipping)	0.2
γ (descuento)	0.99
λ (GAE)	0.95
Pasos por rollout	4 096
Mini-batch size	64
Épocas por actualización	4
Coeficiente entropía	10^{-3}
Coeficiente valor	0.5
Max grad norm	0.5
Target KL	0.02
$\log \sigma_0$ inicial	$-1{,}0$
Optimizador	Adam (eps= 10^{-5})
Pasos totales	20 M

Hiperparámetros principales. Se emplea *learning rate annealing* lineal, reduciendo la tasa de aprendizaje linealmente desde 10^{-4} hasta 0 a lo largo del entrenamiento. Las ventajas se normalizan por rollout (*advantage normalization*), y los retornos objetivo del crítico se normalizan por media y desviación típica para estabilizar las magnitudes.

3.2. Resultados y discusión

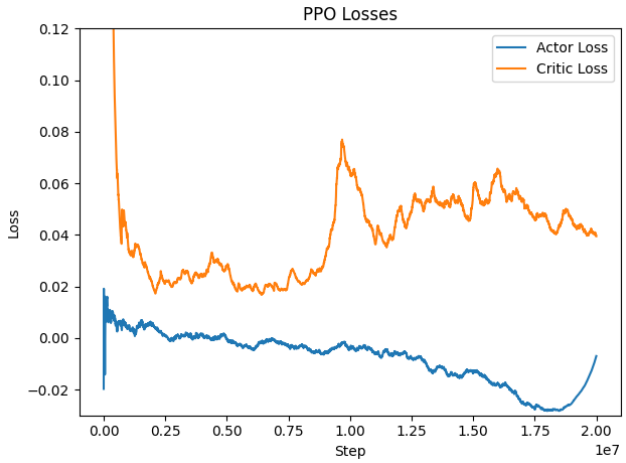


Figura 1: Evolución de las pérdidas del actor y del crítico durante el entrenamiento con PPO.

La gráfica de las pérdidas tanto del actor como del crítico nos permite comentar el primer efecto que observamos y que se extiende a ambos algoritmos: la pérdida del crítico no disminuye y se mantiene estable. Si bien la pérdida del actor sí que disminuye de forma consistente y no parece estar estancada, en este caso ambas pérdidas se mantienen bastante estables.

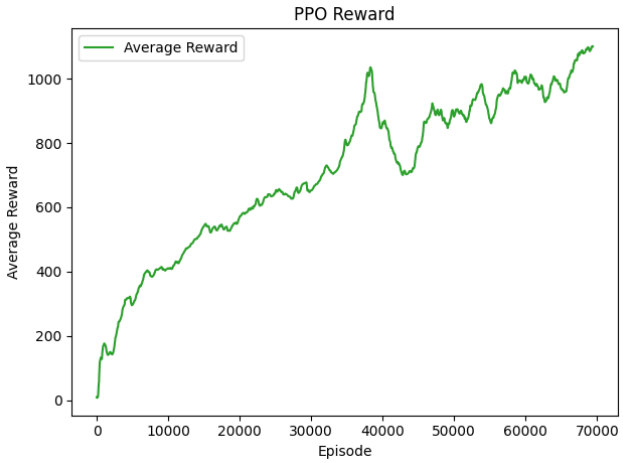


Figura 2: Recompensa episódica media durante el entrenamiento con PPO.

En el caso de la recompensa, observamos que crece de forma constante, indicando entrenamiento. Cabe apuntar que hemos notado que los agentes del entorno del Walker, con la actual función de recompensa, no han conseguido andar hasta no alcanzar una recompensa media de alrededor de 1000.

4. Parte 2: Locomoción con SAC

4.1. Metodología

SAC maximiza el objetivo de máxima entropía:

$$J(\pi) = \mathbb{E}\left[\sum_t r(s_t, a_t) + \alpha\, \mathcal{H}(\pi(\cdot|s_t))\right] \tag{5}$$

El algoritmo incorpora los siguientes componentes clave:

- **Replay buffer uniforme** de capacidad 100 000 con muestreo aleatorio.
- **Doble critic** (twin Q-networks) con *target networks* actualizadas mediante Polyak averaging: $\theta_{\text{target}} \leftarrow \tau\theta + (1-\tau)\theta_{\text{target}}$
- **Reparameterization trick**: $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \cdot \varepsilon)$, $\varepsilon \sim \mathcal{N}(0, I)$
- **Corrección de log-probabilidad** por la transformación tanh:

$$\log \pi(a|s) = \log \pi(u|s) - \sum_i \log(1 - \tanh^2(u_i) + \epsilon) \tag{6}$$

Se realiza un ajuste automático de α (SAC-v2): se optimiza $\log \alpha$ con el objetivo $L(\alpha) = -\alpha\,(\log \pi(a|s) + \bar{\mathcal{H}})$, donde $\bar{\mathcal{H}} = -N_a$ es la entropía objetivo.

Las pérdidas del crítico se calculan como:

$$L_Q = \frac{1}{2}\left[\text{MSE}(Q_1, y) + \text{MSE}(Q_2, y)\right] \tag{7}$$

con target $y = r + \gamma(1-d)[\text{mín}(Q_1', Q_2') - \alpha \log \pi(a'|s')]$, y la pérdida del actor como:

$$L_\pi = \mathbb{E}_s[\alpha \log \pi(a|s) - \text{mín}(Q_1, Q_2)] \tag{8}$$

Durante la actualización del actor, los parámetros del crítico se congelan temporalmente (`requires_grad = False`) para evitar que los gradientes del actor corrompan los pesos del crítico.

Arquitectura. A diferencia de PPO, SAC utiliza redes separadas para actor y crítico:

- **Actor** (`ContinuousActor`): Encoder CNN (Nature-DQN, idéntico al de PPO) $\rightarrow$ `Linear`($n_{\text{flat}} \rightarrow 512$) $\rightarrow$ ReLU, con dos cabezas lineales: μ (`std=0.01`) y $\log \sigma$ (`std=1.0`), acotado a $[-5{,}0, 2{,}0]$. Las acciones se mapean al rango $[\text{low}, \text{high}]$ mediante $a = \tanh(u) \cdot s + b$.
- **Critic** (`ContinuousCritic`): Dos redes Q completamente independientes, cada una con su propio encoder CNN. Las features del estado (512 dim) se concatenan con el vector de acción (6 dim) y se procesan por una cabeza MLP: `Linear`($518 \rightarrow 256$) $\rightarrow$ ReLU $\rightarrow$ `Linear`($256 \rightarrow 1$).
- **Target critic**: Copia del critic, sin gradientes, actualizada con Polyak.

Cuadro 2: Hiperparámetros de SAC empleados en el entrenamiento.

Hiperparámetro	Valor
Tasa de aprendizaje (actor)	3×10^{-4}
Tasa de aprendizaje (critic)	3×10^{-4}
Tasa de aprendizaje (α)	3×10^{-4}
γ (descuento)	0.99
τ (Polyak)	0.005
Tamaño replay buffer	100 000
Tamaño batch	256
α_0 (entropía inicial)	0.2
Entropía objetivo	$-N_a = -6$
Pasos de calentamiento	1 000 (acciones aleatorias)
Frecuencia de entrenamiento	cada 4 pasos
Max grad norm	10.0
Optimizador	Adam (eps= 10^{-4})
Pasos totales	20 M

Hiperparámetros principales. Esta tabla representa la configuración de hiperparámetros usada para entrenar el modelo de SAC con el que logramos mejores resultados. Su optimización se hizo a mano por prueba y error. Se hará referencia a este modelo como experimento original o base en comparaciones más adelante.

4.2. Resultados y discusión

En esta sección discutimos los resultados de SAC en el entrenamiento que mejores resultados ha presentado.

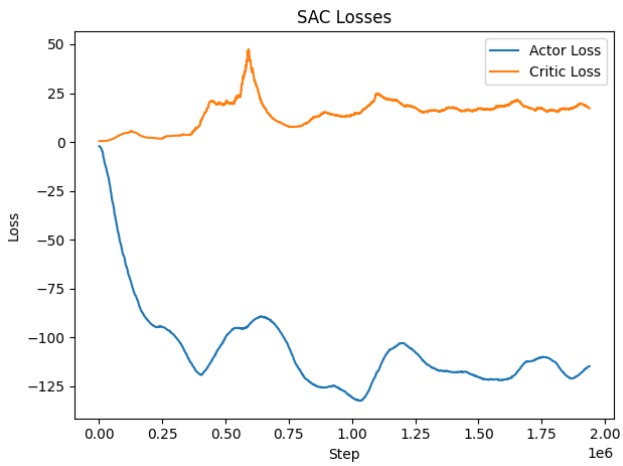


Figura 3: Evolución de las pérdidas del actor y del crítico durante el entrenamiento con SAC.

En esta gráfica podemos observar las pérdidas del algoritmo a lo largo del entrenamiento. Igual que con PPO, observamos que la pérdida del actor decrece uniformemente, indicando aprendizaje, mientras que la del crítico se estanca y se mantiene estable (lo cual no impide el entrenamiento).

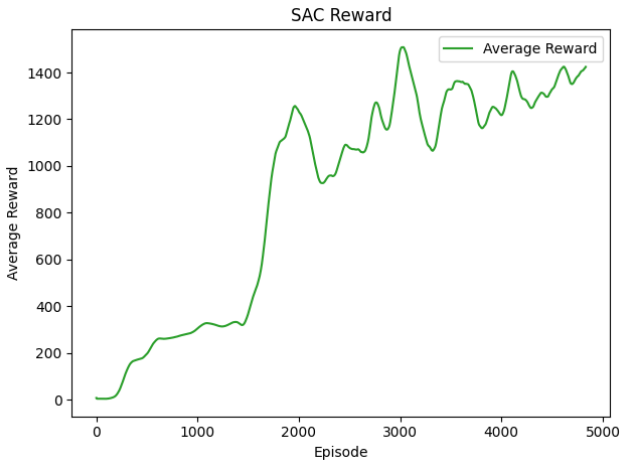


Figura 4: Recompensa episódica media durante el entrenamiento con SAC.

Observamos que la gráfica de la recompensa evoluciona de forma estable, superando el valor de 1000 y aprendiendo a andar.

5. Parte 3: Comparativa PPO vs. SAC

5.1. Velocidad de aprendizaje

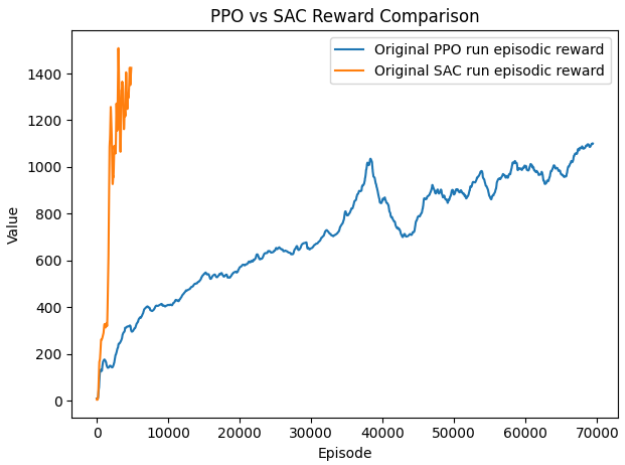


Figura 5: Comparativa de la evolución de la recompensa episódica media de PPO y SAC.

Para comparar la velocidad de aprendizaje, es especialmente útil recurrir a las gráficas de recompensas por episodios. Observando la representación vemos que el algoritmo de SAC llega a muchos menos episodios que el de PPO. Esto está directamente relacionado con lo rápido que se observa que SAC llega a recompensas superiores a 1000 en comparación con PPO.

El entrenamiento de ambos modelos está limitado por número de steps, no por número de episodios. Como SAC supera ese umbral de recompensa antes de los 5 000 episodios y ya sabe andar, a partir de ese momento cada episodio empieza a durar más steps. De esta forma, para el mismo número de steps, como SAC aprende más rápido, SAC consigue finalizar menos episodios que PPO.

5.2. Estabilidad

Antes de comparar la estabilidad de los algoritmos analizando las gráficas, es importante recordar que dichas representaciones están suavizadas fuertemente para que sean informativas, ya que los datos contienen mucho ruido (excepto en el caso de las recompensas medias).

Con esto en cuenta, podemos empezar a analizar la estabilidad de los algoritmos. En el caso de PPO, si bien la pérdida del actor desciende de forma estable, al compararla con la pérdida del actor de SAC vemos que esta última es mucho menor. Esto parece estar directamente relacionado con la diferencia de velocidad entre los algoritmos, además de que SAC aprende a desplazarse mejor, consiguiendo episodios de media más largos.

A parte de esto, ambos algoritmos muestran estancamiento en la pérdida de su crítico y un crecimiento estable de sus gráficas de recompensa.

5.3. Rendimiento final

El análisis de rendimiento lo realizamos de forma cualitativa en función de los vídeos generados durante la evaluación.

Como indican las gráficas de pérdida y de recompensa, SAC aprende mejor que PPO, y esto se refleja directamente en el rendimiento final. SAC consigue episodios más largos y con mejor técnica, mientras que PPO sigue fallando en algunos episodios incluso en las etapas más avanzadas del entrenamiento.

Como ya hemos comentado, ambos modelos son capaces de andar; sin embargo, ninguno ha conseguido la calidad de desplazamiento que observábamos en prácticas anteriores con el algoritmo de RAINBOW o DQN. Esto lo achacamos principalmente al tiempo de entrenamiento y al espacio de acciones. Al ser un espacio continuo, entendemos que los modelos anden de formas menos humanas, ya que se rigen por una función de recompensa bastante laxa y encontrar una forma humana de andar es más complicado si no se les provee de un set de acciones orientadas como cuando se discretizan las acciones. Por otro lado, creemos que con suficiente tiempo de entrenamiento acabarían igualando o superando a DQN y RAINBOW, especialmente SAC que presenta una velocidad de aprendizaje muy alta.

5.4. Sensibilidad a hiperparámetros: análisis con Optuna

Para cuantificar la sensibilidad a los hiperparámetros, se realizaron experimentos de optimización bayesiana con **Optuna**. Se lanzaron $N_{\text{PPO}} = 10$ y $N_{\text{SAC}} = 3$ *trials* respectivamente, sobre entrenamientos reducidos de 5 000 pasos por trial (con 1 000 pasos de warm-up en SAC) para limitar el coste computacional. La métrica optimizada es el **retorno episódico medio de los últimos 5 episodios** completos de cada trial, es decir,

la suma total de recompensas por episodio promediada sobre los últimos 5 episodios finalizados.

A continuación se presentan los resultados de la búsqueda, contrastados con los resultados obtenidos en los experimentos largos presentados anteriormente. Las gráficas están suavizadas para facilitar su interpretación. Además, se debe tener en cuenta que esta búsqueda de hiperparámetros se realizó después del entrenamiento exitoso de los modelos (resultados anteriores).

Cuadro 3: Espacio de búsqueda de hiperparámetros empleado en Optuna para PPO y SAC.

Algoritmo	Hiperparámetro	Rango mínimo	Rango máximo
PPO	lr	10^{-5}	3×10^{-4}
	γ	0.95	0.999
	λ_{GAE}	0.9	0.98
	ϵ_{clip}	0.1	0.3
	coef. entropía	10^{-4}	0.05
SAC	lr	10^{-5}	3×10^{-4}
	γ	0.95	0.999
	τ	0.005	0.02
	α_0	10^{-3}	0.2

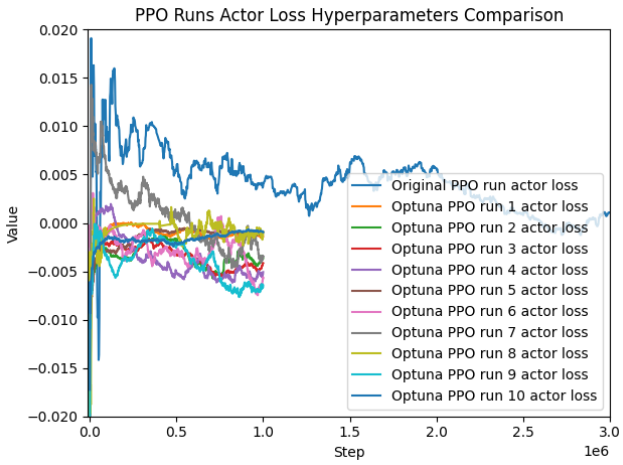
5.4.1. Resultados PPO

Se completaron 10 trials, con retornos episódicos medios oscilando entre 69.3 y 540.4.

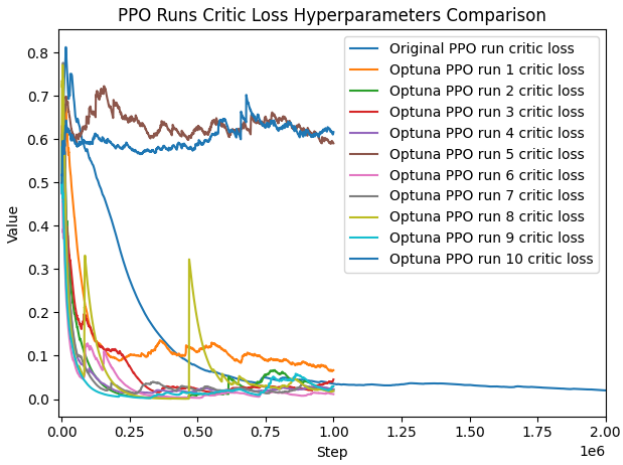
Cuadro 4: Resumen de los 10 trials de Optuna para PPO (ordenados por recompensa).

Trial	lr	γ	λ	ϵ	Retorno
7	$2{,}58 \times 10^{-4}$	0.952	0.927	0.166	**540.4**
6	$6{,}17 \times 10^{-5}$	0.963	0.921	0.250	456.2
1	$1{,}27 \times 10^{-5}$	0.994	0.956	0.109	451.0
4	$1{,}16 \times 10^{-4}$	0.960	0.926	0.108	353.9
8	$1{,}06 \times 10^{-4}$	0.972	0.904	0.290	326.3
10	$4{,}66 \times 10^{-5}$	0.973	0.960	0.151	277.0
2	$2{,}63 \times 10^{-5}$	0.980	0.933	0.166	275.9
3	$2{,}27 \times 10^{-5}$	0.980	0.958	0.154	241.3
5	$5{,}50 \times 10^{-5}$	0.954	0.976	0.232	153.6
9	$5{,}26 \times 10^{-5}$	0.970	0.951	0.222	69.3

El mejor trial (#7) alcanzó un retorno episódico medio de 540.4 con lr $= 2{,}58 \times 10^{-4}$, $\gamma = 0{,}952$, $\lambda = 0{,}927$, $\epsilon = 0{,}166$ y un coeficiente de entropía muy bajo ($1{,}01 \times 10^{-4}$). Se observa que los trials con tasas de aprendizaje más altas ($> 10^{-4}$) tienden a obtener mejores resultados, mientras que valores de γ más bajos ($\approx 0{,}95$) parecen favorecer el aprendizaje inicial frente a descuentos elevados ($\gamma > 0{,}98$). Valores de λ_{GAE} moderados ($\approx 0{,}92$) superan a los más altos, sugiriendo que una estimación de ventaja con menor varianza es preferible en las primeras etapas.



(a) Pérdida del actor.



(b) Pérdida del crítico.

Figura 6: Comparativa de las pérdidas del actor (a) y del crítico (b) entre los trials de Optuna para PPO. Cada curva corresponde a un trial con hiperparámetros distintos.

La gráfica de las pérdidas del actor muestra un comportamiento homogéneo en todos los trials: la variación es poca pero se mantiene un descenso estable. En el caso de la pérdida del crítico, si bien vemos que la mayoría siguen la tendencia vista en la Figura 1, hay un par de trials que se estancan al menos en los steps iniciales del entrenamiento.

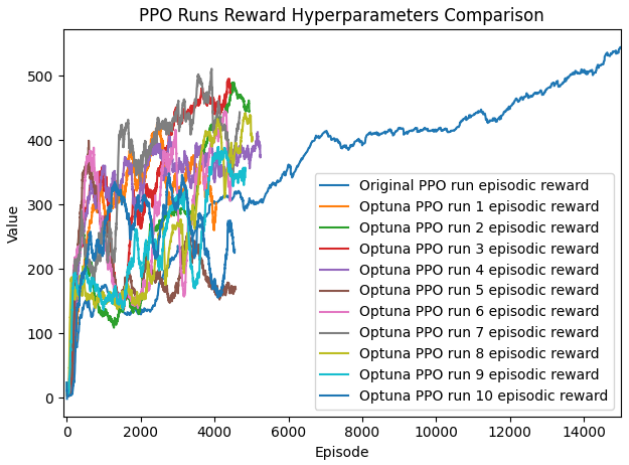


Figura 7: Comparativa de la recompensa episódica entre los trials de Optuna para PPO.

En cuanto a las funciones de recompensa, vemos un crecimiento bastante similar entre todos los trials, aunque la tendencia de los trials cuyas pérdidas del crítico eran más altas se sitúan por debajo de las demás. En general, parece que, con el tiempo, cualquier conjunto de los hiperparámetros probados crecería establemente como el trial original.

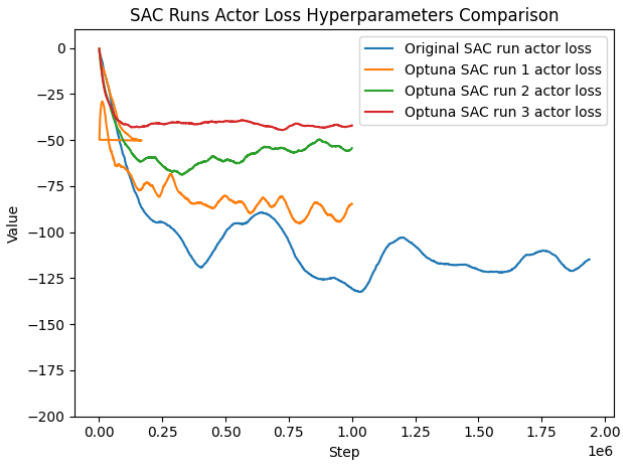
5.4.2. Resultados SAC

Se completaron 3 trials, con una variabilidad notable. El mejor trial (#1) obtuvo un retorno episódico medio de 856.2, más del doble que los otros dos trials.

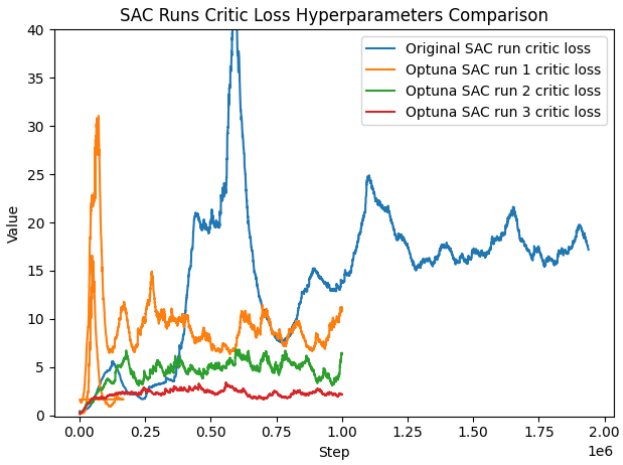
Cuadro 5: Resumen de los 3 trials de Optuna para SAC.

Trial	lr	γ	τ	α_0	Retorno
1	$3{,}51\times10^{-5}$	0.985	0.015	0.189	**856.2**
3	$1{,}79\times10^{-5}$	0.969	0.020	0.037	409.7
2	$1{,}67\times10^{-5}$	0.974	0.015	0.002	386.6

El trial ganador se distingue por un $\alpha_0 = 0{,}189$ relativamente alto, lo que sugiere que una exploración inicial vigorosa es crucial para SAC en las fases tempranas. Los trials con valores de α_0 muy bajos ($< 0{,}04$) obtuvieron recompensas notablemente inferiores, indicando una alta sensibilidad de SAC a la temperatura de entropía inicial. Los valores de τ y γ mostraron menor influencia dentro de los rangos explorados.



(a) Pérdida del actor.



(b) Pérdida del crítico.

Figura 8: Comparativa de las pérdidas del actor (a) y del crítico (b) entre los trials de Optuna para SAC. Cada curva corresponde a un trial con hiperparámetros distintos.

Las métricas de pérdida en el estudio de hiperparámetros para SAC son muy homogéneas, manteniendo todas un comportamiento muy similar. La pérdida del actor decrece bastante, mientras que la del crítico se mantiene en un rango estrecho de valores sin llegar a descender claramente.

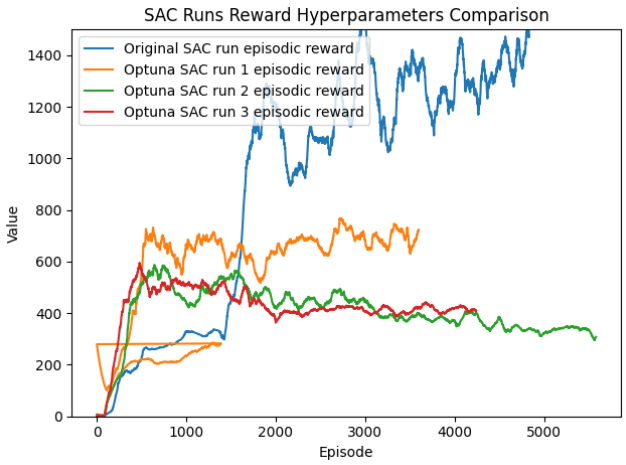


Figura 9: Comparativa de la recompensa episódica entre los trials de Optuna para SAC.

En el caso de las recompensas se observa un comportamiento extraño. Mientras que los trials de optuna crecen de forma moderada o incluso se estancan hasta el final de sus episodios, los resultados del experimento original presentan un salto muy alto en la recompensa alrededor de los 1500 episodios. Cabe entonces preguntarse si el resto de conjuntos de hiperparámetros presentarían el mismo comportamiento de haber continuado entrenando.

6. Conclusiones generales

A lo largo de esta práctica se han implementado y evaluado dos algoritmos Actor-Critic para control continuo con observaciones RGB: PPO (on-policy) y SAC (off-policy). De los resultados obtenidos se extraen las siguientes conclusiones:

- **Pérdidas**: las funciones de pérdida del actor disminuyen de forma consistente en ambos algoritmos, reflejando aprendizaje efectivo de la política. Sin embargo, la pérdida del crítico tiende a estabilizarse sin descender o incluso a aumentar ligeramente, un comportamiento observado tanto en los entrenamientos largos como en los trials de Optuna.
- **Eficiencia muestral**: SAC demuestra una eficiencia muestral superior a PPO, consiguiendo aprender a andar con menos episodios gracias al reuso de transiciones mediante su replay buffer.
- **Sensibilidad a hiperparámetros**: el análisis con Optuna revela que SAC es especialmente sensible al valor inicial de α (temperatura de entropía), mientras que PPO muestra un comportamiento más homogéneo entre distintas configuraciones. La falta de tiempo computacional impidió probar los hiperparámetros más prometedores en entrenamientos largos completos.
- **Rendimiento final**: SAC supera a PPO tanto en recompensa acumulada como en calidad del desplazamiento observada cualitativamente. No obstante, ninguno de los dos algoritmos alcanza la calidad de locomoción observada con DQN/RAINBOW en prácticas anteriores, lo que atribuimos a la mayor complejidad del espacio de acciones continuo y al tiempo limitado de entrenamiento.